

Simulação Artificial de Evolução Neuronal

F. Bottega

17 de junho de 2026

Resumo

Este artigo descreve um modelo computacional de organismos digitais controlados por cérebros artificiais evolutivos. Diferentemente de redes neurais artificiais tradicionais baseadas em camadas, o modelo proposto representa cada cérebro como uma malha tridimensional de neurônios parcialmente conectados, com topologia, parâmetros de disparo, regiões de conectividade e regras de plasticidade definidos por um genoma discreto. Os organismos habitam um ambiente bidimensional, recebem sinais sensoriais, processam tais sinais por meio de uma dinâmica neuronal recorrente e executam ações motoras binárias. A seleção não é definida por uma função de perda explícita; ela emerge de uma ecologia simples baseada em energia, alimentação, sobrevivência e reprodução. São apresentadas a formalização do genoma, as regras de geração fenotípica, a dinâmica de propagação de sinais, a plasticidade das conexões, o mecanismo de recombinação por offspring e a metodologia de implementação em Godot e Python via WebSocket. Além da descrição formal, o texto enfatiza uma apresentação didática do sistema por meio de fluxogramas, diagramas e tabelas, de modo a tornar explícitos os vínculos entre teoria, algoritmo e implementação.

Palavras-chave: neuroevolução; algoritmos genéticos; agentes artificiais; plasticidade neuronal; simulação baseada em agentes; sistemas complexos.

1 Introdução

Redes neurais artificiais modernas são, em geral, construídas a partir de arquiteturas previamente especificadas: camadas, blocos, funções de ativação, operações matriciais e uma função objetivo que guia o ajuste de parâmetros. Esta abordagem é extraordinariamente eficaz em inúmeros domínios, mas impõe uma estrutura computacional fortemente projetada. Mesmo quando a arquitetura é grande e o treinamento é estatístico, boa parte da forma do sistema cognitivo é definida antes da experiência.

O objetivo deste trabalho é explorar uma direção complementar: simular organismos digitais cujas estruturas neuronais não sejam organizadas em camadas, mas em uma malha tridimensional parcialmente conectada, geneticamente especificada e plasticamente modificável ao longo da vida. A hipótese metodológica não é que tal modelo seja imediatamente competitivo com redes neurais artificiais convencionais, mas que ele oferece um espaço experimental adequado para investigar formas mais orgânicas de adaptação, recorrência, memória, auto-organização e seleção ambiental.

Neste modelo, um organismo digital, chamado aqui de *blob*, possui um cérebro artificial. O cérebro é definido por um tensor discreto

$$B = [a] \times [b] \times [c],$$

em que $[n] = \{0, 1, \dots, n - 1\}$ e $a, b, c \in \mathbb{N}$. Cada elemento de B representa um neurônio. Alguns neurônios recebem sinais sensoriais, alguns estão associados a ações motoras e os demais funcionam como neurônios internos. As conexões entre neurônios são direcionais, não necessariamente simétricas, e não seguem uma estrutura de camadas.

A contribuição deste texto é formalizar o protótipo atual da simulação. O artigo descreve:

1. a representação genética usada para gerar cérebros artificiais;
2. a transformação genótipo–fenótipo;
3. a dinâmica de sinais neuronais;
4. as regras de plasticidade estrutural e de enfraquecimento de conexões;
5. as regras de offspring por recombinação genética;
6. a metodologia de implementação em frontend Godot e backend Python.

A simulação atual deve ser entendida como um protótipo exploratório. Não há, nesta versão, uma função fitness escalar global, retropropagação de gradiente ou aprendizado supervisionado. A pressão seletiva é ambiental: organismos que consomem energia sem encontrar comida morrem; organismos que sobrevivem tempo suficiente e colidem com outros organismos podem gerar descendentes.

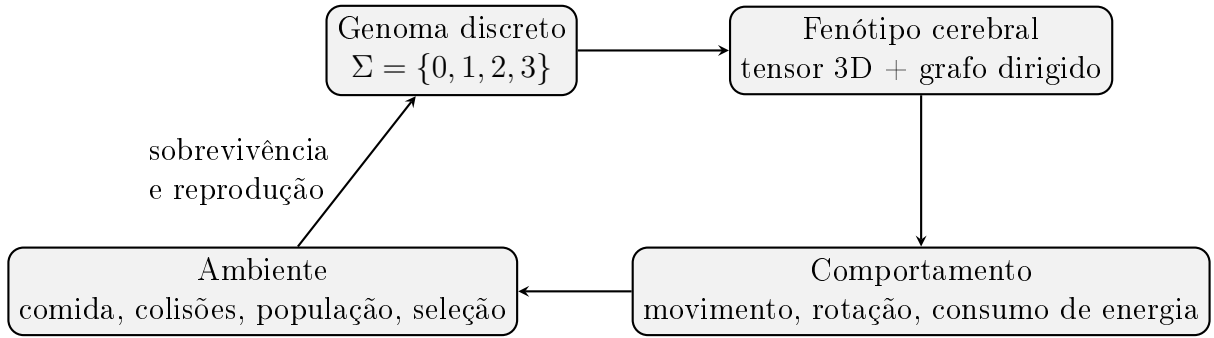


Figura 1: Ciclo conceitual do modelo: o genoma define o cérebro, o cérebro produz comportamento, o comportamento interage com o ambiente e o ambiente determina quais genomas persistem.

2 Definições iniciais

2.1 Espaço neuronal

Fixadas dimensões a, b, c , o conjunto de neurônios é

$$V_{a,b,c} = [a] \times [b] \times [c].$$

Para fins de implementação, cada neurônio $(i, j, k) \in V_{a,b,c}$ é identificado por um índice linear

$$\alpha(i, j, k) = i + ja + kab.$$

O conjunto de índices neuronais é, portanto,

$$\mathcal{V}_{a,b,c} = \{0, 1, \dots, abc - 1\}.$$

A transformação inversa é dada por

$$x(n) = n \bmod a, \quad y(n) = \left\lfloor \frac{n}{a} \right\rfloor \bmod b, \quad z(n) = \left\lfloor \frac{n}{ab} \right\rfloor.$$

Assim, quando conveniente, identificamos o neurônio $n \in \mathcal{V}_{a,b,c}$ com a coordenada

$$\alpha^{-1}(n) = (x(n), y(n), z(n)).$$

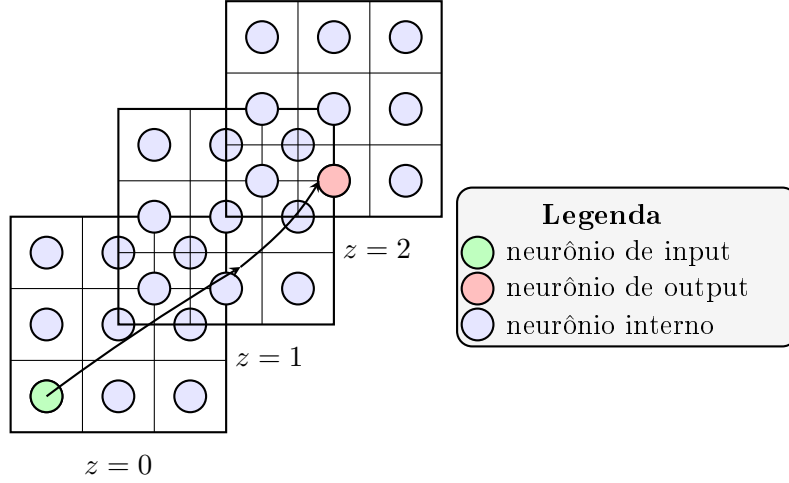


Figura 2: Representação didática de um cérebro como tensor tridimensional. Em cada plano z há uma malha de neurônios; alguns neurônios são especializados como inputs e outputs, enquanto os demais são internos. As setas ilustram conexões direcionais.

2.2 Grafo cerebral

O cérebro de um organismo é representado por um grafo direcionado

$$G = (\mathcal{V}, E),$$

em que $\mathcal{V} = \mathcal{V}_{a,b,c}$ e

$$E \subseteq \mathcal{V} \times \mathcal{V}.$$

Uma aresta $(u, v) \in E$ indica que o neurônio u envia sinal ao neurônio v . O modelo assume que, para qualquer par $\{u, v\}$, não devem coexistir as duas direções:

$$(u, v) \in E \implies (v, u) \notin E.$$

Esta restrição torna a conexão direcional e evita conexões recíprocas imediatas.

Cada conexão (u, v) possui uma força sináptica

$$\varepsilon_{uv}(t) \in \mathbb{R}_{\geq 0}.$$

No nascimento do organismo, todas as conexões existentes iniciam com força

$$\varepsilon_{uv}(0) = 1.$$

2.3 Inputs, outputs e sinais

Sejam N_I o número de inputs e N_O o número de outputs. Os neurônios de input formam uma lista

$$I = (i_1, \dots, i_{N_I}) \subset \mathcal{V},$$

e os neurônios de output formam uma lista

$$O = (o_1, \dots, o_{N_O}) \subset \mathcal{V}.$$

A geração genética exige que:

$$i_p \neq i_q \quad (p \neq q), \quad o_p \neq o_q \quad (p \neq q), \quad I \cap O = \emptyset.$$

Logo, cada input é associado a exatamente um neurônio, cada output é associado a exatamente um neurônio, e nenhum neurônio pode ser simultaneamente input e output.

O estado neuronal no instante t é uma função esparsa

$$s_t : \mathcal{V} \rightarrow \mathbb{R}_{\geq 0},$$

na qual $s_t(u)$ é o sinal armazenado no neurônio u . Na implementação, apenas neurônios com algum sinal já registrado precisam aparecer explicitamente no dicionário de sinais.

A função de saturação usada para normalizar sinais é

$$\sigma_\kappa(x) = 1 - e^{-x/\kappa},$$

em que $\kappa \in \{1, 2, 3, 4\}$ é um parâmetro geneticamente definido. Note que, para $x \geq 0$, temos $0 \leq \sigma_\kappa(x) < 1$.

Tabela 1: Objetos básicos do modelo e sua interpretação.

Objeto	Notação	Interpretação
Cérebro	$B = [a] \times [b] \times [c]$	malha tridimensional de neurônios
Grafo	$G = (\mathcal{V}, E)$	estrutura efetiva de conectividade dirigida
Inputs	$I = (i_1, \dots, i_{N_I})$	neurônios que recebem sinais sensoriais
Outputs	$O = (o_1, \dots, o_{N_O})$	neurônios que determinam ações motoras
Sinal neuronal	$s_t(u)$	intensidade armazenada no neurônio u no instante t
Força sináptica	$\varepsilon_{uv}(t)$	intensidade da conexão dirigida (u, v)
Threshold interno	θ_u	limiar de disparo do neurônio u
Threshold de output	$\Theta_{o_r}^{out}$	limiar para ativar a ação associada a o_r

3 Representação genética

3.1 Alfabeto genético

O genoma é definido sobre o alfabeto discreto

$$\Sigma = \{0, 1, 2, 3\}.$$

Opcionalmente, estes símbolos podem ser interpretados como

$$0 \mapsto A, \quad 1 \mapsto T, \quad 2 \mapsto C, \quad 3 \mapsto G.$$

Essa interpretação é apenas simbólica. Na implementação, os genes são manipulados como inteiros em $\{0, 1, 2, 3\}$.

Dada uma palavra genética

$$w = (w_1, \dots, w_m) \in \Sigma^m,$$

define-se a conversão decimal

$$D_m(w) = \sum_{\ell=1}^m w_\ell 10^{m-\ell}.$$

Observe que D_m não interpreta a palavra como número em base 4. Ela concatena os símbolos como dígitos decimais restritos a 0, 1, 2, 3. Por exemplo,

$$D_5(1, 0, 3, 2, 1) = 10321.$$

Também será usada a aplicação

$$\mu : \Sigma \rightarrow \{0.2, 0.4, 0.6, 0.8\}, \quad \mu(q) = \frac{q+1}{5}.$$

3.2 Parâmetros globais

A implementação atual usa os parâmetros constantes da Tabela 2.

Tabela 2: Parâmetros globais do modelo atual.

Símbolo	Valor	Significado
N_I	16	número de inputs sensoriais
N_O	4	número de outputs motores
D	5	tamanho de palavra para coordenadas de input/output
D_2	2	tamanho de palavra para regiões e coordenadas locais
D_3	6	tamanho de palavra para o decay
D_4	4	tamanho de palavra para número inicial de conexões
$A_{\min}, A_{\max}$	4, 7	limites da primeira dimensão cerebral
$B_{\min}, B_{\max}$	4, 7	limites da segunda dimensão cerebral
$C_{\min}, C_{\max}$	4, 7	limites da terceira dimensão cerebral

3.3 Genoma completo

O genoma de um organismo recém-gerado pode ser escrito como

$$\mathcal{G} = (G_{\text{dim}}, G_I, G_O, G_{O,\theta}, G_R, G_Q, G_E, G_P, G_\theta, G_\delta, G_\tau, G_\kappa, G_\omega).$$

Cada componente codifica uma parte do fenótipo cerebral:

Tabela 3: Componentes genéticos e respectivos fenótipos.

Gene	Forma	Fenótipo associado
G_{dim}	$\Sigma^{3 \times D}$	dimensões (a, b, c)
G_I	$\Sigma^{N_I \times 3D}$	neurônios de input
G_O	$\Sigma^{N_O \times 3D}$	neurônios de output
$G_{O,\theta}$	Σ^{N_O}	thresholds dos outputs
G_R	$\Sigma^{abc \times 3D_2}$	regiões locais de conectividade
G_Q	Σ^{D_4}	número máximo de candidatos iniciais por neurônio
G_E	dicionário de palavras em Σ^{3D_2}	candidatos a conexões iniciais
G_P	Σ^{abc}	probabilidades de criação de novas conexões
G_θ	Σ^{abc}	thresholds de disparo neuronal
G_δ	Σ^{D_3}	decaimento de força sináptica
G_τ	Σ^{D_2}	período de inatividade pós-disparo
G_κ	Σ^{D_2}	fator de saturação da resposta exponencial
G_ω	Σ^{D_2}	gene reservado para período enfraquecido

O gene G_ω é preservado na especificação porque já existe na implementação, mas seu fenótipo ainda não é usado na dinâmica atual.

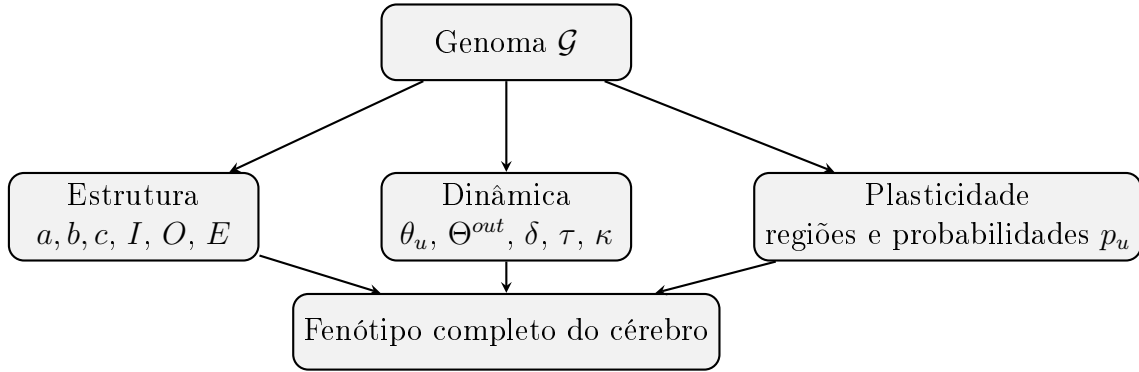


Figura 3: Decomposição didática do genoma em três grupos funcionais: genes estruturais, genes dinâmicos e genes de plasticidade.

4 Regras genéticas de geração do cérebro

Esta seção descreve a construção de um cérebro a partir de um genoma aleatório. A geração é rejeitada e repetida quando as restrições estruturais de inputs e outputs não são satisfeitas.

4.1 Dimensões

Sejam

$$G_{\text{dim}} = \begin{pmatrix} g_1^a & \cdots & g_D^a \\ g_1^b & \cdots & g_D^b \\ g_1^c & \cdots & g_D^c \end{pmatrix}.$$

As dimensões do cérebro são

$$\begin{aligned} a &= \max\{A_{\min}, D_D(g_1^a, \dots, g_D^a) \bmod (A_{\max} + 1)\}, \\ b &= \max\{B_{\min}, D_D(g_1^b, \dots, g_D^b) \bmod (B_{\max} + 1)\}, \\ c &= \max\{C_{\min}, D_D(g_1^c, \dots, g_D^c) \bmod (C_{\max} + 1)\}. \end{aligned}$$

Na configuração atual, $a, b, c \in \{4, 5, 6, 7\}$ e o número de neurônios é

$$M = abc.$$

4.2 Mapeamento genético para coordenadas

Dada uma palavra

$$w = (w^x, w^y, w^z) \in \Sigma^{3m},$$

com blocos $w^x, w^y, w^z \in \Sigma^m$, define-se

$$\Phi_{a,b,c}^{(m)}(w) = \alpha(D_m(w^x) \bmod a, D_m(w^y) \bmod b, D_m(w^z) \bmod c).$$

Esta função transforma uma palavra genética em um índice neuronal.

4.3 Neurônios de input

Para cada input $r \in \{1, \dots, N_I\}$, a linha $G_I[r]$ define

$$i_r = \Phi_{a,b,c}^{(D)}(G_I[r]).$$

O conjunto/lista de inputs só é aceito se

$$\text{card}\{i_1, \dots, i_{N_I}\} = N_I.$$

Se dois inputs forem mapeados para o mesmo neurônio, o cérebro é rejeitado e um novo genoma é gerado.

4.4 Neurônios de output

Analogamente, para cada output $r \in \{1, \dots, N_O\}$,

$$o_r = \Phi_{a,b,c}^{(D)}(G_O[r]).$$

Os outputs são aceitos apenas se

$$\text{card}\{o_1, \dots, o_{N_O}\} = N_O$$

e

$$\{o_1, \dots, o_{N_O}\} \cap \{i_1, \dots, i_{N_I}\} = \emptyset.$$

Se alguma destas condições falhar, o cérebro é rejeitado.

4.5 Thresholds de output

Cada neurônio de output recebe um threshold

$$\Theta_{o_r}^{out} = \mu(G_{O,\theta}[r]).$$

Logo,

$$\Theta_{o_r}^{out} \in \{0.2, 0.4, 0.6, 0.8\}.$$

Na leitura dos outputs, o output r será ativado se

$$\sigma_\kappa(s(o_r)) > \Theta_{o_r}^{out}.$$

4.6 Regiões locais de conectividade

Cada neurônio $u = \alpha(i, j, k)$ possui uma região retangular local determinada geneticamente. Seja $G_R[u] \in \Sigma^{3D_2}$ dividido em três blocos. Define-se

$$R_x(u) = D_{D_2}(G_R^x[u]) \bmod a,$$

$$R_y(u) = D_{D_2}(G_R^y[u]) \bmod b,$$

$$R_z(u) = D_{D_2}(G_R^z[u]) \bmod c.$$

A região nominal de conectividade de u é

$$\mathcal{R}(u) = \{\alpha(p, q, r) : |p - i| \leq R_x(u), |q - j| \leq R_y(u), |r - k| \leq R_z(u)\},$$

com as coordenadas restritas aos limites do tensor cerebral.

A interpretação biológica aproximada é que cada neurônio possui um alcance local próprio. Não há, portanto, um raio global fixo para todo o cérebro.

4.7 Conexões iniciais

O gene $G_Q \in \Sigma^{D_4}$ determina o número de candidatos iniciais por neurônio:

$$Q = D_{D_4}(G_Q) \bmod (M - 1).$$

Para cada neurônio $u = \alpha(i, j, k)$, são geradas Q palavras candidatas

$$G_E[u, 1], \dots, G_E[u, Q] \in \Sigma^{3D_2}.$$

Cada candidata define um possível destino

$$v_w = \Phi_{a,b,c}^{(D_2)}(G_E[u, w]).$$

A conexão (u, v_w) é adicionada se, e somente se, as quatro condições abaixo forem satisfeitas:

$$v_w \notin E(u),$$

$$(v_w, u) \notin E,$$

$$v_w \in \mathcal{R}(u),$$

$u \neq v_w$ ou, caso se admita auto-conexão, ela não viole as demais restrições.

Na implementação atual, a verificação explícita impede duplicidade e conexão reversa; auto-conexões não são destacadas como um caso especial. A estrutura efetiva é o conjunto de conexões aceitas após essas filtragens.

4.8 Probabilidade de plasticidade estrutural

Cada neurônio recebe uma probabilidade de criação de novas conexões:

$$p_u = \mu(G_P[u]) \in \{0.2, 0.4, 0.6, 0.8\}.$$

Esse parâmetro é usado durante a vida do organismo, quando o neurônio dispara e tenta criar uma nova conexão com outro neurônio ativo em sua região.

4.9 Threshold de disparo neuronal

Cada neurônio recebe um threshold de disparo:

$$\theta_u = \mu(G_\theta[u]) \in \{0.2, 0.4, 0.6, 0.8\}.$$

Um neurônio u só propaga sinal quando

$$s_t(u) > \theta_u$$

e quando não está em período de inatividade.

4.10 Decaimento sináptico

O gene de decay é uma palavra

$$G_\delta = (d_1, \dots, d_{D_3}) \in \Sigma^{D_3}.$$

Define-se

$$\delta = \sum_{\ell=1}^{D_3} d_\ell 10^{-\ell}.$$

Equivalentemente, δ é o número decimal $0.d_1d_2\dots d_{D_3}$. Como $d_\ell \in \{0, 1, 2, 3\}$, no protótipo atual δ está no intervalo

$$0 \leq \delta \leq 0.333333.$$

A cada atualização, conexões internas podem ter sua força reduzida por

$$\delta^2.$$

4.11 Inatividade pós-disparo

O período de inatividade é

$$\tau = D_{D_2}(G_\tau) + 1.$$

Este valor define o contador usado para impedir que um neurônio dispare continuamente em iterações consecutivas. Após disparar, o neurônio entra em um estado refratário controlado por esse contador.

4.12 Fator exponencial

O parâmetro da função de saturação é

$$\kappa = \left\lfloor \frac{D_{D_2}(G_\kappa)}{10} \right\rfloor + 1.$$

Como $D_{D_2}(G_\kappa)$ varia de 0 a 33, segue que

$$\kappa \in \{1, 2, 3, 4\}.$$

4.13 Gene reservado para enfraquecimento temporário

O gene

$$G_\omega \in \Sigma^{D_2}$$

define, na implementação atual, o valor

$$\omega = \tau + D_{D_2}(G_\omega) + 1.$$

Entretanto, ω ainda não participa da dinâmica de sinais. Ele é mantido como parâmetro reservado para uma possível extensão em que neurônios recém-disparados tenham um período enfraquecido distinto do período inativo.

4.14 Algoritmo de geração genética

Algoritmo 1. Geração de um cérebro novo

1. Amostrar todos os genes de $\mathcal{G}$ de forma independente e uniforme em Σ .
2. Calcular (a, b, c) a partir de G_{dim} .
3. Construir os neurônios de input I a partir de G_I .
4. Se houver colisão entre inputs, rejeitar o genoma e reiniciar.
5. Construir os neurônios de output O a partir de G_O .
6. Se houver colisão entre outputs, rejeitar o genoma e reiniciar.
7. Se $I \cap O \neq \emptyset$, rejeitar o genoma e reiniciar.
8. Calcular thresholds de output, regiões locais, número inicial de candidatos, conexões iniciais, probabilidades de plasticidade, thresholds internos, decay, inatividade e fator exponencial.
9. Inicializar todos os sinais neuronais como vazios.
10. Inicializar todas as forças sinápticas existentes como 1.
11. Inicializar o contador de inatividade de cada neurônio com valor τ .
12. Retornar o genoma simbólico $\mathcal{G}$ e o fenótipo cerebral resultante.

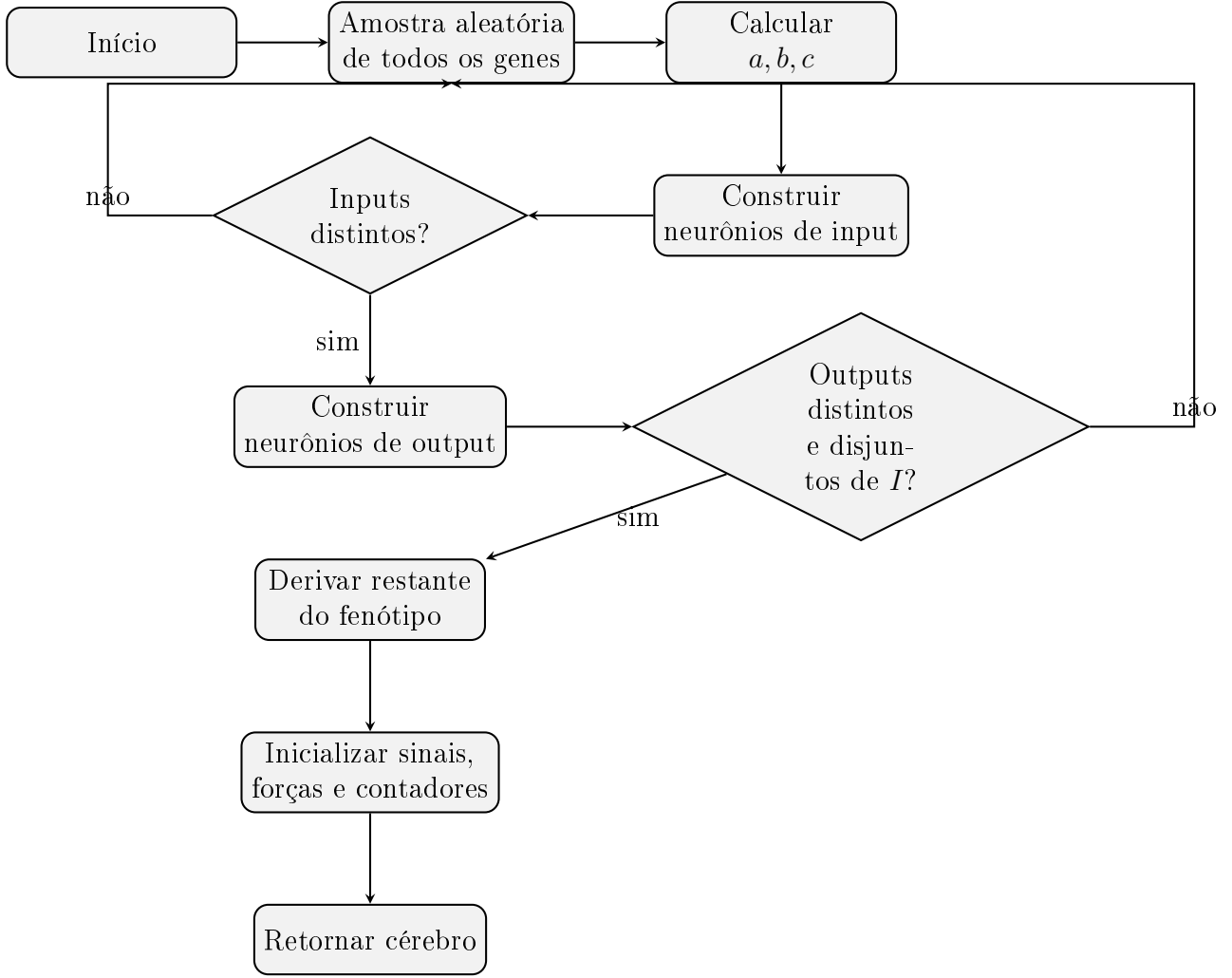


Figura 4: Fluxograma do Algoritmo 1. A geração genética é um procedimento de amostragem com rejeição: sempre que há colisão entre inputs ou inválida separação input/output, o genoma é descartado e um novo é amostrado.

5 Dinâmica neuronal

5.1 Estrutura de estado

Para cada organismo, a simulação mantém:

- o conjunto de conexões E_t ;
- as forças sinápticas $\varepsilon_{uv}(t)$;
- os sinais neuronais $s_t(u)$;
- os contadores de inatividade $c_t(u)$;
- o número corrente de conexões.

O processamento de um passo neuronal recebe um vetor de inputs externos

$$X_t = (X_t^1, \dots, X_t^{N_I}) \in \mathbb{R}_{\geq 0}^{N_I}$$

e retorna um vetor binário de outputs

$$Y_t = (Y_t^1, \dots, Y_t^{No}) \in \{0, 1\}^{No}.$$

5.2 Propagação entre neurônios

Considere um neurônio u com sinal $s_t(u) > 0$. Ele dispara se

$$s_t(u) > \theta_u$$

e

$$c_t(u) = \tau.$$

Quando dispara, o sinal efetivamente enviado é

$$\hat{s}_t(u) = \sigma_\kappa(s_t(u)).$$

Para cada conexão $(u, v) \in E_t$, define-se a força atualizada por uso:

$$\tilde{\varepsilon}_{uv}(t) = \frac{1 + \varepsilon_{uv}(t)}{2}.$$

Então

$$\varepsilon_{uv}(t) \leftarrow \tilde{\varepsilon}_{uv}(t)$$

e o neurônio v recebe contribuição

$$\Delta s(v) \leftarrow \Delta s(v) + \tilde{\varepsilon}_{uv}(t) \hat{s}_t(u).$$

Após transmitir, o neurônio u é marcado para zeragem:

$$s_{t+1}(u) \leftarrow 0,$$

salvo se receber outro sinal agregado no mesmo passo.

5.3 Plasticidade estrutural

Após um disparo, o neurônio u tenta criar uma nova conexão. Seja

$$\mathcal{C}_t(u) \subseteq \mathcal{R}(u)$$

um conjunto de candidatos amostrados dentro da região local de u . Na implementação, por economia computacional, a busca percorre uma amostra de aproximadamente metade dos pontos de cada eixo da região, em vez de percorrer a região completa.

Para cada candidato $v \in \mathcal{C}_t(u)$, uma nova conexão (u, v) é criada se:

$$U \leq p_u, \quad U \sim \text{Uniforme}(0, 1),$$

$$v \in \text{dom}(s_t),$$

$$s_t(v) > 0,$$

$$(u, v) \notin E_t,$$

$$(v, u) \notin E_t.$$

Quando tais condições são satisfeitas,

$$E_{t+1} \leftarrow E_t \cup \{(u, v)\},$$

$$\varepsilon_{uv}(t+1) = 1.$$

A regra cria no máximo uma nova conexão por disparo de neurônio.

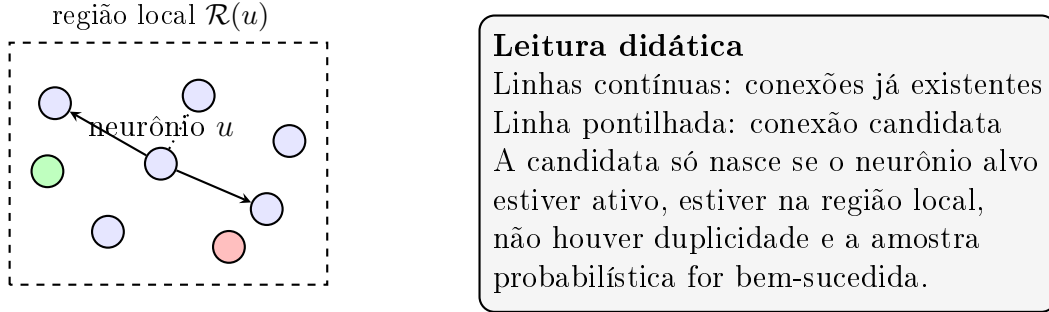


Figura 5: Esquema da plasticidade estrutural local. Um neurônio ativo tenta criar uma nova conexão apenas dentro de sua região genética de conectividade.

5.4 Atualização do contador de inatividade

Quando um neurônio dispara, seu contador é reduzido. Enquanto o contador for diferente de τ , ele continua sendo decrementado a cada passo. Se o contador se torna negativo, ele é restaurado para τ . Em forma algorítmica:

$$c(u) \leftarrow c(u) - 1 \quad \text{no disparo,}$$

e, ao final da avaliação do neurônio,

$$c(u) \neq \tau \implies c(u) \leftarrow c(u) - 1,$$

$$c(u) < 0 \implies c(u) \leftarrow \tau.$$

Esse mecanismo funciona como um período refratário discreto. A duração efetiva depende de τ e da ordem exata de atualização.

5.5 Enfraquecimento de conexões

Após a tentativa de disparo e de plasticidade estrutural, as conexões de u são enfraquecidas. A regra atual preserva todas as conexões que envolvem neurônios de input ou output. Portanto, uma conexão (u, v) só sofre decay se

$$u \notin I \cup O \quad \text{e} \quad v \notin I \cup O.$$

Nesse caso,

$$\varepsilon_{uv}(t+1) = \varepsilon_{uv}(t) - \delta^2.$$

Se

$$\varepsilon_{uv}(t+1) < 0,$$

a conexão é removida:

$$E_{t+1} \leftarrow E_{t+1} \setminus \{(u, v)\}.$$

Essa regra implementa uma competição estrutural: conexões internas pouco usadas tendem a desaparecer, enquanto conexões usadas têm sua força elevada em direção a 1 pela regra

$$\varepsilon \mapsto \frac{1 + \varepsilon}{2}.$$

5.6 Injeção dos inputs externos

Depois da propagação interna, os inputs externos são injetados nos neurônios de input. Para cada $r \in \{1, \dots, N_I\}$,

$$s(i_r) \leftarrow \sigma_\kappa(X_t^r).$$

Assim, sinais sensoriais são normalizados pela mesma resposta exponencial usada na dinâmica interna.

5.7 Leitura dos outputs

Após a injeção dos inputs, os outputs são lidos. Para cada $r \in \{1, \dots, N_O\}$,

$$Y_t^r = \begin{cases} 1, & \text{se } o_r \in \text{dom}(s) \text{ e } \sigma_\kappa(s(o_r)) > \Theta_{o_r}^{\text{out}}, \\ 0, & \text{caso contrário.} \end{cases}$$

Na implementação atual há quatro outputs motores:

Tabela 4: Interpretação dos outputs no ambiente bidimensional.

Output	Ação
Y^1	movimento para frente na direção da rotação atual
Y^2	movimento para trás em relação à rotação atual
Y^3	rotação positiva
Y^4	rotação negativa

5.8 Algoritmo de processamento neuronal

Algoritmo 2. Processamento de um organismo em um passo

1. Receber o vetor de inputs externos X_t .
2. Criar um acumulador temporário Δs vazio.
3. Criar uma lista Z de neurônios que serão zerados após disparar.
4. Para cada neurônio u atualmente presente no dicionário de sinais:
 - (a) Se $s(u) > \theta_u$ e $c(u) = \tau$, então:
 - i. calcular $\widehat{s}(u) = \sigma_\kappa(s(u))$;
 - ii. adicionar u à lista Z ;
 - iii. para cada conexão (u, v) , atualizar $\varepsilon_{uv} \leftarrow (1 + \varepsilon_{uv})/2$ e acumular $\Delta s(v) \leftarrow \Delta s(v) + \varepsilon_{uv}\widehat{s}(u)$;
 - iv. tentar criar uma nova conexão partindo de u .

- (b) Atualizar o contador de inatividade de u .
 - (c) Aplicar decay às conexões internas de u .
5. Para cada $u \in Z$, definir $s(u) \leftarrow 0$.
 6. Para cada v em Δs , definir $s(v) \leftarrow \Delta s(v)$.
 7. Injetar os inputs externos nos neurônios $i_1, \dots, i_{N_I}$.
 8. Ler os outputs $o_1, \dots, o_{N_O}$.
 9. Retornar o vetor binário Y_t .

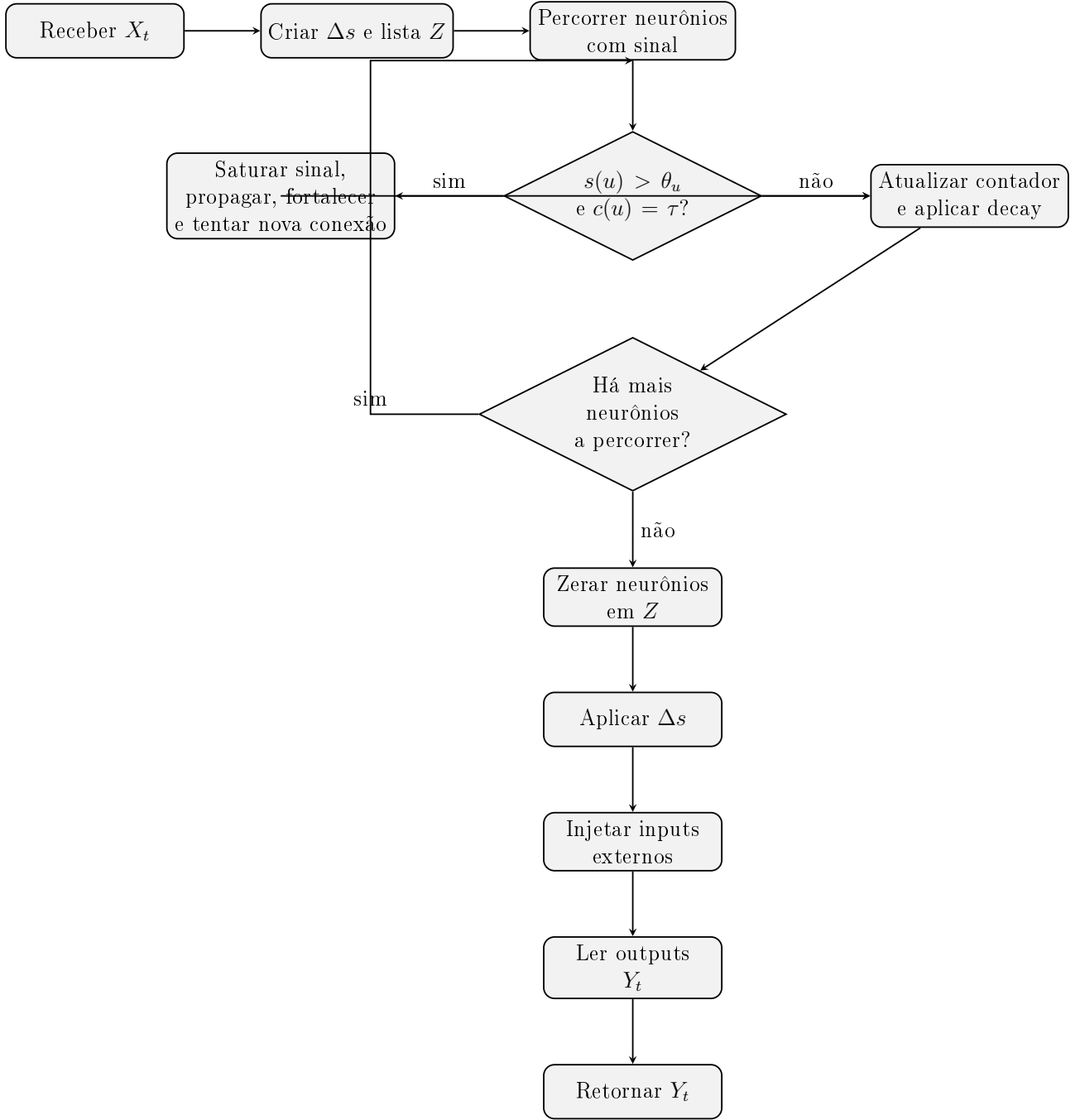


Figura 6: Fluxograma do Algoritmo 2. O processamento neuronal combina atualização interna, plasticidade, injeção sensorial e leitura de outputs em um único passo discreto.

6 Regras de offspring

6.1 Evento reprodutivo

A reprodução é disparada pelo ambiente quando dois blobs colidem e satisfazem as condições comportamentais impostas pela simulação: idade mínima, limite de filhos, cooldown e limite populacional. Quando a reprodução é solicitada, o backend recebe os identificadores de dois organismos pais, digamos P_1 e P_2 .

Se algum dos pais não estiver registrado no conjunto de organismos vivos mantido pelo

backend, o offspring falha. Caso contrário, tenta-se gerar um filho por recombinação genética. No protótipo atual, a geração é tentada no máximo duas vezes; se as restrições de validade falharem repetidamente, nenhum filho é retornado.

6.2 Dimensões do filho

Sejam

$$(a_1, b_1, c_1), \quad (a_2, b_2, c_2)$$

as dimensões dos pais. As dimensões do filho são amostradas uniformemente nos intervalos fechados entre os valores parentais:

$$a_h \sim \text{UnifInt}(\min(a_1, a_2), \max(a_1, a_2)),$$

$$b_h \sim \text{UnifInt}(\min(b_1, b_2), \max(b_1, b_2)),$$

$$c_h \sim \text{UnifInt}(\min(c_1, c_2), \max(c_1, c_2)).$$

O número de neurônios do filho é

$$M_h = a_h b_h c_h.$$

6.3 Operador de corte

Para recombinar blocos de tamanho L , define-se um ponto de corte

$$\rho_L \sim \text{UnifInt}(\lfloor 0.4L \rfloor, \lfloor 0.6L \rfloor).$$

A operação de crossover por corte é

$$\text{cross}(A, B; \rho_L)_r = \begin{cases} A_r, & r < \rho_L, \\ B_r, & r \geq \rho_L. \end{cases}$$

A implementação usa esse padrão para genes de input, output, thresholds de output, regiões, conexões iniciais, probabilidades e thresholds internos, com adaptações quando o filho possui mais neurônios que os pais.

6.4 Inputs do filho

Escolhe-se

$$\rho_I \sim \text{UnifInt}(\lfloor 0.4N_I \rfloor, \lfloor 0.6N_I \rfloor).$$

O gene de inputs do filho é

$$G_I^h[r] = \begin{cases} G_I^{P_1}[r], & r < \rho_I, \\ G_I^{P_2}[r], & r \geq \rho_I. \end{cases}$$

Em seguida, cada linha é reinterpretada nas dimensões do filho:

$$i_r^h = \Phi_{a_h, b_h, c_h}^{(D)}(G_I^h[r]).$$

A geração é aceita apenas se

$$\text{card}\{i_1^h, \dots, i_{N_I}^h\} = N_I.$$

Caso contrário, uma nova tentativa de offspring é iniciada.

6.5 Outputs do filho

Analogamente,

$$\rho_O \sim \text{UnifInt}([0.4N_O], [0.6N_O]),$$

e

$$G_O^h[r] = \begin{cases} G_O^{P_1}[r], & r < \rho_O, \\ G_O^{P_2}[r], & r \geq \rho_O. \end{cases}$$

Os neurônios de output do filho são

$$o_r^h = \Phi_{a_h, b_h, c_h}^{(D)}(G_O^h[r]).$$

A geração é aceita apenas se

$$\text{card}\{o_1^h, \dots, o_{N_O}^h\} = N_O$$

e

$$\{o_1^h, \dots, o_{N_O}^h\} \cap \{i_1^h, \dots, i_{N_I}^h\} = \emptyset.$$

6.6 Thresholds de output do filho

Escolhe-se

$$\rho_{O,\theta} \sim \text{UnifInt}([0.4N_O], [0.6N_O]).$$

Então

$$G_{O,\theta}^h[r] = \begin{cases} G_{O,\theta}^{P_1}[r], & r < \rho_{O,\theta}, \\ G_{O,\theta}^{P_2}[r], & r \geq \rho_{O,\theta}. \end{cases}$$

O threshold fenotípico é novamente

$$\Theta_{o_r^h}^{\text{out},h} = \mu(G_{O,\theta}^h[r]).$$

6.7 Regiões locais do filho

As regiões são recombinadas sobre M_h posições. Escolhe-se

$$\rho_R \sim \text{UnifInt}([0.4M_h], [0.6M_h]).$$

Para cada índice $u \in \{0, \dots, M_h - 1\}$,

$$G_R^h[u] = \begin{cases} G_R^{P_1}[u], & u < \rho_R \text{ e } u < M_1, \\ G_R^{P_2}[u], & u < \rho_R \text{ e } u \geq M_1 \text{ e } u < M_2, \\ \text{gene aleatório}, & u < \rho_R \text{ e não há gene parental disponível}, \\ G_R^{P_2}[u], & u \geq \rho_R \text{ e } u < M_2, \\ G_R^{P_1}[u], & u \geq \rho_R \text{ e } u \geq M_2 \text{ e } u < M_1, \\ \text{gene aleatório}, & u \geq \rho_R \text{ e não há gene parental disponível}. \end{cases}$$

Isto é: antes do corte há preferência pelo pai P_1 ; depois do corte há preferência pelo pai P_2 ; quando o filho possui neurônios além do alcance genético dos pais, o gene é preenchido aleatoriamente.

6.8 Número inicial de conexões do filho

O gene G_Q^h é escolhido integralmente de um dos pais:

$$G_Q^h = \begin{cases} G_Q^{P_1}, & U > 0.5, \\ G_Q^{P_2}, & U \leq 0.5, \end{cases} \quad U \sim \text{Uniforme}(0, 1).$$

Então

$$Q_h = D_{D_4}(G_Q^h) \bmod (M_h - 1).$$

6.9 Conexões iniciais do filho

Para cada neurônio u do filho, escolhe-se

$$\rho_E \sim \text{UnifInt}(\lfloor 0.4Q_h \rfloor, \lfloor 0.6Q_h \rfloor).$$

Para cada candidato $w \in \{1, \dots, Q_h\}$, usa-se a regra de preferência:

$$G_E^h[u, w] = \begin{cases} G_E^{P_1}[u, w], & w < \rho_E \text{ e disponível,} \\ G_E^{P_2}[u, w], & w < \rho_E \text{ e } P_1 \text{ indisponível, mas } P_2 \text{ disponível,} \\ \text{gene aleatório,} & w < \rho_E \text{ e indisponível nos dois pais,} \\ G_E^{P_2}[u, w], & w \geq \rho_E \text{ e disponível,} \\ G_E^{P_1}[u, w], & w \geq \rho_E \text{ e } P_2 \text{ indisponível, mas } P_1 \text{ disponível,} \\ \text{gene aleatório,} & w \geq \rho_E \text{ e indisponível nos dois pais.} \end{cases}$$

Cada palavra candidata é reinterpretada nas dimensões do filho, e a conexão correspondente é aceita se não for duplicada, se a conexão reversa não existir e se o destino estiver dentro da região local do neurônio de origem.

6.10 Probabilidades de plasticidade do filho

Escolhe-se

$$\rho_P \sim \text{UnifInt}(\lfloor 0.4M_h \rfloor, \lfloor 0.6M_h \rfloor).$$

Para cada neurônio u ,

$$G_P^h[u] = \begin{cases} G_P^{P_1}[u], & u < \rho_P \text{ e disponível,} \\ G_P^{P_2}[u], & u < \rho_P \text{ e } P_1 \text{ indisponível, mas } P_2 \text{ disponível,} \\ \text{gene aleatório,} & u < \rho_P \text{ e indisponível nos dois pais,} \\ G_P^{P_2}[u], & u \geq \rho_P \text{ e disponível,} \\ G_P^{P_1}[u], & u \geq \rho_P \text{ e } P_2 \text{ indisponível, mas } P_1 \text{ disponível,} \\ \text{gene aleatório,} & u \geq \rho_P \text{ e indisponível nos dois pais.} \end{cases}$$

O fenótipo é

$$p_u^h = \mu(G_P^h[u]).$$

6.11 Thresholds internos do filho

A recombinação dos thresholds internos segue a mesma regra das probabilidades:

$$\rho_\theta \sim \text{UnifInt}(\lfloor 0.4M_h \rfloor, \lfloor 0.6M_h \rfloor),$$

com preferência por P_1 antes do corte e por P_2 depois do corte. O threshold final é

$$\theta_u^h = \mu(G_\theta^h[u]).$$

6.12 Parâmetros globais herdados ou gerados

O decay é herdado integralmente de um dos pais:

$$G_\delta^h = \begin{cases} G_\delta^{P_1}, & U > 0.5, \\ G_\delta^{P_2}, & U \leq 0.5. \end{cases}$$

A inatividade pós-disparo é gerada novamente de forma aleatória:

$$G_\tau^h \sim \text{Uniforme}(\Sigma^{D_2}), \quad \tau_h = D_{D_2}(G_\tau^h) + 1.$$

O fator exponencial é herdado integralmente de um dos pais:

$$G_\kappa^h = \begin{cases} G_\kappa^{P_1}, & U > 0.5, \\ G_\kappa^{P_2}, & U \leq 0.5. \end{cases}$$

O gene reservado G_ω também é herdado integralmente de um dos pais:

$$G_\omega^h = \begin{cases} G_\omega^{P_1}, & U > 0.5, \\ G_\omega^{P_2}, & U \leq 0.5. \end{cases}$$

6.13 Algoritmo de offspring

Algoritmo 3. Geração de descendente

1. Receber identificadores dos pais P_1 e P_2 .
2. Se algum pai não existir no backend, retornar falha.
3. Repetir por no máximo duas tentativas:
 - (a) amostrar dimensões do filho entre as dimensões dos pais;
 - (b) recombinar genes de input por corte entre 40% e 60%;
 - (c) reinterpretar inputs nas dimensões do filho;
 - (d) se houver colisão entre inputs, reiniciar tentativa;
 - (e) recombinar genes de output por corte entre 40% e 60%;
 - (f) reinterpretar outputs nas dimensões do filho;
 - (g) se houver colisão entre outputs ou interseção input–output, reiniciar tentativa;
 - (h) recombinar thresholds de output;

- (i) recombinar regiões locais, usando genes aleatórios quando o filho possui neurônios sem correspondente parental;
 - (j) herdar de um dos pais o gene do número inicial de conexões;
 - (k) recombinar os genes de conexões iniciais para cada neurônio;
 - (l) recombinar probabilidades de plasticidade;
 - (m) recombinar thresholds internos;
 - (n) herdar decay, fator exponencial e gene reservado de enfraquecimento;
 - (o) gerar aleatoriamente o período de inatividade;
 - (p) inicializar sinais, forças sinápticas, contadores e número de filhos.
4. Se alguma tentativa for aceita, retornar o filho.
 5. Se todas as tentativas falharem, retornar falha.

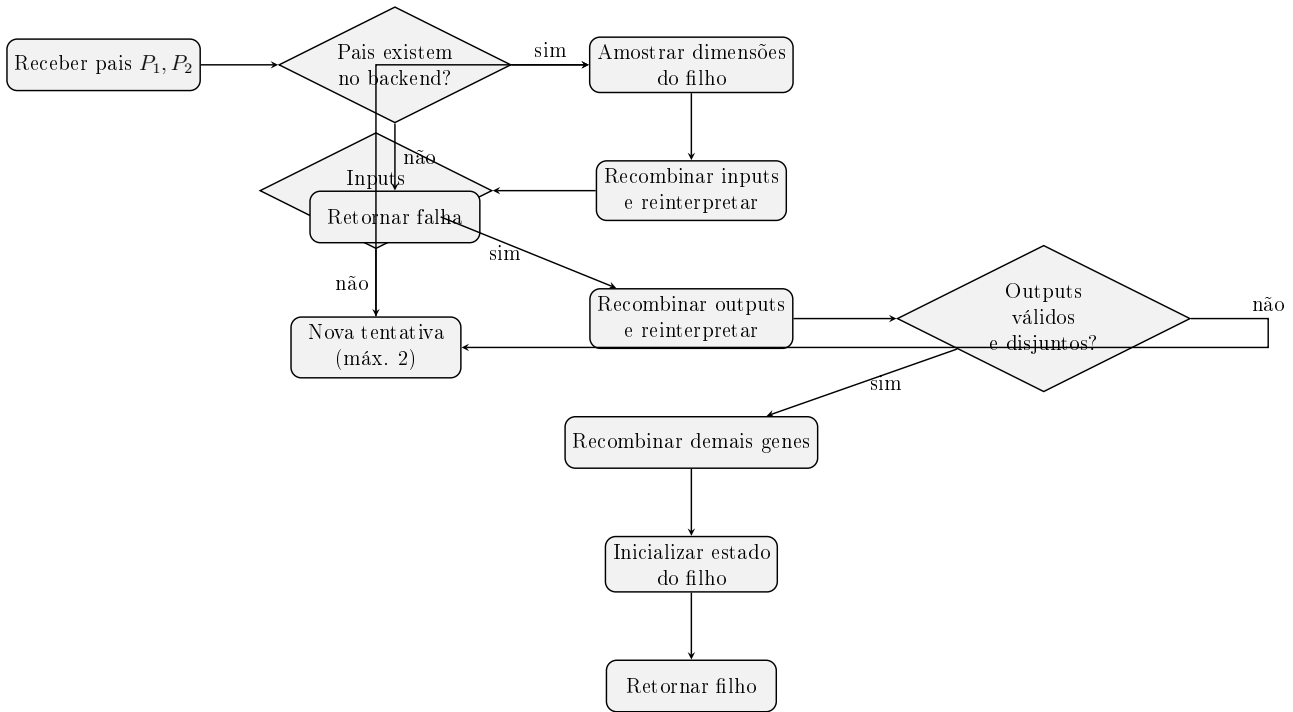


Figura 7: Fluxograma do Algoritmo 3. O offspring é uma recombinação com validação estrutural; quando inputs ou outputs do filho se tornam inválidos, uma nova tentativa é executada.

7 Ecologia, energia e pressão seletiva

O ambiente atua como mecanismo de seleção. Cada organismo possui uma energia $E_b(t)$, inicializada em 1.0. A execução de ações consome energia:

Tabela 5: Custos energéticos das ações implementadas.

Ação	Condição	Custo
movimento para frente	$Y^1 = 1$	0.002
movimento para trás	$Y^2 = 1$ e $Y^1 \neq 1$	0.002
rotação positiva	$Y^3 = 1$	0.0015
rotação negativa	$Y^4 = 1$ e $Y^3 \neq 1$	0.0015
decaimento basal	todo passo processado	0.001
reprodução	evento de offspring	0.01

Quando o organismo come, sua energia aumenta:

$$E_b(t+1) = \min\{2.0, E_b(t) + 0.5\}.$$

Quando sua energia fica negativa, o organismo é removido da cena. Assim, a sobrevivência depende de produzir ações que levem o blob a encontrar comida antes que a energia seja consumida.

A reprodução exige colisão entre dois blobs e as seguintes condições: idade mínima superior a 100 passos de vida, cooldown de reprodução, limite máximo de filhos por blob e população total inferior ao máximo permitido. O nascimento do filho ocorre com energia inicial 0.5.

Não há fitness explícita. A aptidão é implícita e ecológica:

$$\text{aptidão efetiva} \approx \text{sobreviver} + \text{encontrar comida} + \text{reproduzir}.$$

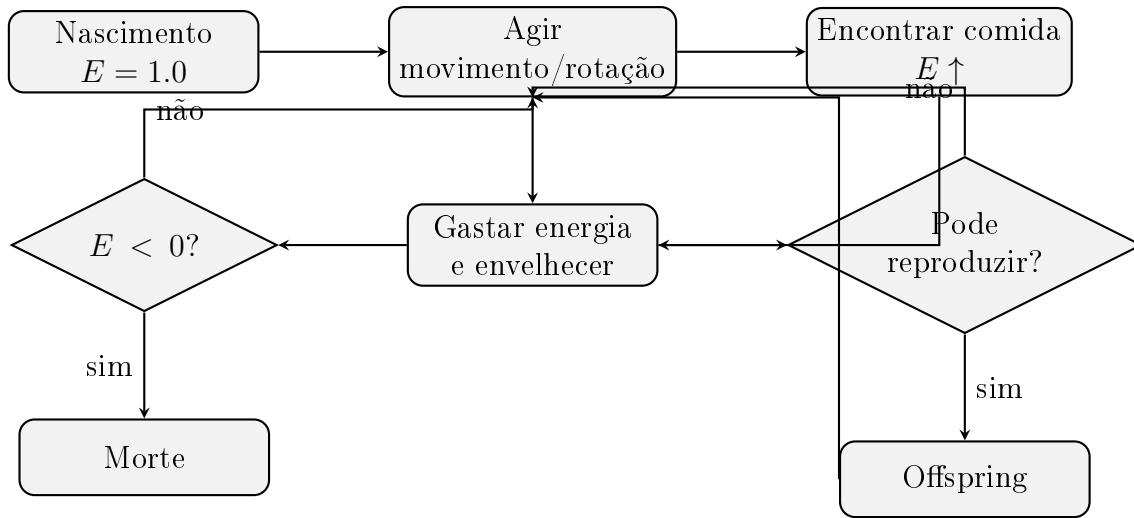


Figura 8: Ciclo ecológico simplificado do blob: agir, gastar energia, encontrar comida e eventualmente reproduzir ou morrer.

8 Metodologia da implementação

8.1 Arquitetura geral

A implementação é dividida em duas partes:

1. **Frontend em Godot:** responsável por ambiente bidimensional, física, visualização, colisões, comida, energia, coleta de inputs e aplicação de outputs.
2. **Backend em Python:** responsável por geração dos cérebros, armazenamento dos genomas, processamento neuronal, plasticidade, offspring e comunicação via WebSocket.

A comunicação ocorre por WebSocket local no endereço

`ws://127.0.0.1:8000/ws`

O backend usa FastAPI e Uvicorn. O frontend usa `WebSocketPeer` do Godot.

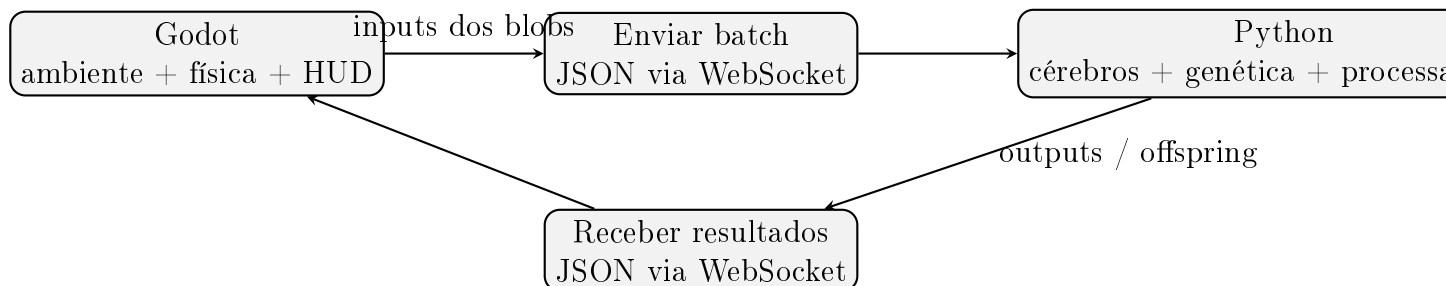


Figura 9: Arquitetura lógico-operacional da implementação. O Godot controla o ambiente e o Python controla a lógica cerebral; ambos se comunicam por lotes via WebSocket.

8.2 Protocolo de comunicação

A comunicação é feita por lotes. O frontend envia uma mensagem JSON da forma

```
{"batch": [requisições]}
```

Cada requisição possui um campo `npc_action`. As ações principais são:

Tabela 6: Ações suportadas pelo backend.

Ação	Função
<code>gen_brain</code>	gerar novo cérebro para um blob
<code>process</code>	processar inputs e retornar outputs
<code>offspring</code>	gerar descendente a partir de dois pais

A resposta do backend tem a forma

```
{"results": [resultados]}
```

Para processamento, cada resultado inclui identificador do blob, vetor de outputs, número corrente de conexões e tamanho máximo possível do cérebro.

8.3 Inputs implementados

O vetor de inputs do blob possui 16 componentes:

$$X = (X_1, \dots, X_{16}).$$

Na implementação atual:

$$\begin{aligned} X_1 &= \max(0, v_x), & X_2 &= \max(0, -v_x), \\ X_3 &= \max(0, v_y), & X_4 &= \max(0, -v_y), \\ X_5 &= \max(0, \omega), & X_6 &= \max(0, -\omega), \\ X_7 &= |\rho/(2\pi)|, & X_8 &= E_b, \\ X_9 &= \frac{1}{1 + E_b}, & X_{10} &= \text{contador de proximidade de comida}, \\ X_{11} &= \text{indicador de colisão}, & X_{12} &= \min(10, 100/d), \\ X_{13} &= Y_{t-1}^1, & X_{14} &= Y_{t-1}^2, \\ X_{15} &= Y_{t-1}^3, & X_{16} &= Y_{t-1}^4. \end{aligned}$$

Aqui v_x, v_y são componentes da velocidade linear, ω é a velocidade angular, ρ é a rotação atual do blob, E_b é a energia e d é a distância até uma comida detectada.

Os quatro outputs anteriores são realimentados como inputs, produzindo uma memória comportamental de curto prazo no acoplamento organismo–ambiente.

Tabela 7: Resumo didático dos 16 inputs.

Input	Descrição
X_1, X_2	velocidade horizontal positiva e negativa
X_3, X_4	velocidade vertical positiva e negativa
X_5, X_6	velocidade angular positiva e negativa
X_7	rotação normalizada
X_8, X_9	energia e inverso regularizado da energia
X_{10}, X_{12}	proximidade e distância inversa até comida
X_{11}	colisão
$X_{13}, \dots, X_{16}$	outputs do passo anterior

8.4 Outputs implementados

Os outputs binários controlam corpo e energia:

1. $Y^1 = 1$: define velocidade linear para frente na direção da rotação.
2. $Y^2 = 1$: define velocidade linear para trás em relação à rotação.
3. $Y^3 = 1$: define rotação angular positiva.
4. $Y^4 = 1$: define rotação angular negativa.

O corpo físico do blob é um `RigidBody2D` sem gravidade, com amortecimento linear e angular. A cena contém paredes estáticas que delimitam o ambiente.

8.5 Comida e sensores ambientais

Cada comida possui duas áreas de detecção:

1. uma área maior, que indica proximidade e fornece distância;
2. uma área menor, que representa consumo da comida.

Ao entrar na área maior, o blob recebe sinal de proximidade e distância inversa. Ao entrar na área menor, a comida é removida da cena e a energia do blob aumenta. Novas comidas são adicionadas periodicamente ao ambiente.

8.6 Economia computacional

Processar todos os cérebros em todos os frames pode se tornar custoso. Por isso, o backend usa uma estratégia de economia: o blob de identificador `blob_0` é sempre processado, mas os demais são processados em subconjuntos definidos pelo resto do identificador numérico módulo 5. Assim, cada requisição processa apenas parte da população, reduzindo o custo por frame.

O `blob_0` também é usado como organismo de referência para registro de atividade neuronal. No frontend, sua energia é mantida em 1.0 para impedir que ele morra antes do encerramento da simulação.

8.7 Históricos e visualização

O backend mantém em memória dois históricos:

1. histórico de sinais neuronais;
2. histórico do número de conexões por blob.

O script de visualização posterior usa Plotly para gerar uma animação tridimensional da atividade neuronal e uma série temporal do número de conexões. Esses instrumentos são importantes para avaliar a evolução estrutural e dinâmica dos cérebros, ainda que a versão atual do protótipo deva garantir consistência entre o formato do histórico salvo e o formato esperado pela visualização.

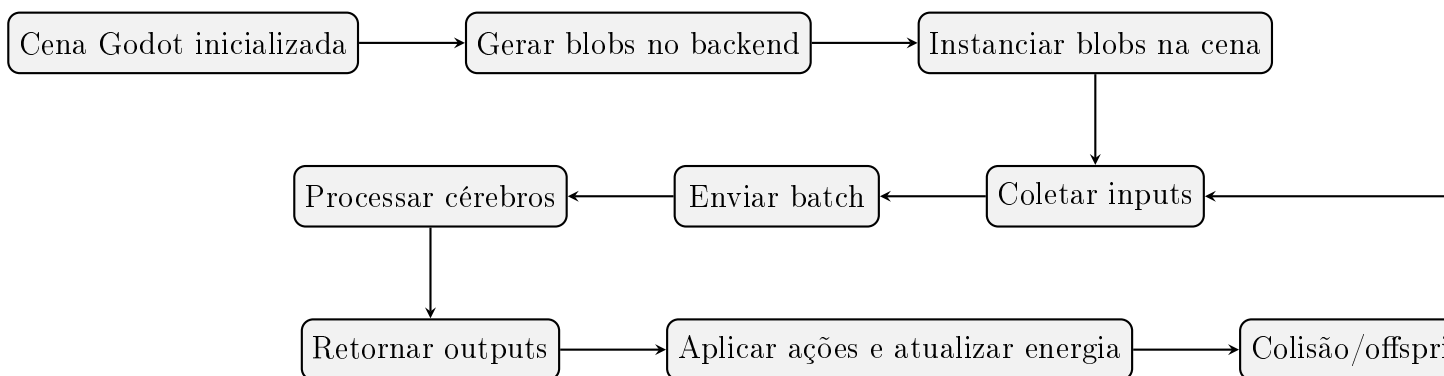


Figura 10: Ciclo operacional resumido da implementação. O sistema alterna entre percepção, processamento cerebral, atuação e atualização ambiental.

9 Discussão

O modelo proposto separa claramente três níveis de adaptação:

1. **Adaptação genética:** estruturas cerebrais são geradas e recombinadas por genes discretos.
2. **Adaptação individual:** conexões internas podem ser criadas e enfraquecidas durante a vida do organismo.
3. **Adaptação ecológica:** o ambiente seleciona organismos por sobrevivência, alimentação e reprodução.

A ausência de camadas e de função de perda explícita torna o modelo menos controlável, mas também mais aberto. Comportamentos úteis não são impostos por engenharia direta; devem emergir de combinações entre topologia, sinais, plasticidade e pressão seletiva. Essa abertura é precisamente o ponto de interesse do protótipo.

Há, entretanto, limitações importantes. A simulação ainda não possui métricas consolidadas de desempenho populacional, mecanismos explícitos de mutação pontual em offspring, análise estatística de linhagens ou comparação com baselines. Além disso, alguns genes já existem na estrutura, como o período enfraquecido, mas ainda não são usados na dinâmica. Tais elementos devem ser tratados como extensões naturais do modelo.

Tabela 8: Síntese das hipóteses e implicações do protótipo.

Hipótese de modelagem	Implicação experimental
Topologia não baseada em camadas	permite recorrência e geometria local emergentes
Genoma discreto com mapeamento indireto	favorece diversidade estrutural e rejeição de cérebros inválidos
Plasticidade durante a vida	separa adaptação ontogenética de adaptação evolutiva
Seleção sem fitness explícita	desloca o problema para a ecologia do ambiente
Processamento distribuído em backend	viabiliza cérebros mais caros do que a execução integral no engine gráfico

10 Conclusão

Este artigo formalizou um protótipo de simulação artificial de evolução neuronal. Cada organismo possui um cérebro definido por uma malha tridimensional de neurônios, conexões direcionais, thresholds, regiões locais de conectividade, resposta exponencial, plasticidade estrutural e enfraquecimento seletivo de conexões internas. A geração inicial dos cérebros é genética, a reprodução ocorre por recombinação de genes parentais e a seleção é determinada pelo ambiente.

O modelo não pretende substituir redes neurais artificiais tradicionais, mas explorar um regime alternativo de inteligência artificial baseada em organismos, ecologia, plasticidade e evolução. O próximo passo natural é instrumentar melhor os históricos, introduzir métricas de linhagem, testar diferentes regras de mutação e avaliar se estruturas neuronais mais adaptadas emergem de forma robusta sob diferentes configurações ambientais.